

QUOTA — Requirements and Environment

Rule: **never commit secrets**. Put real values in `.env` (gitignored). The repo carries only `.env.example` with placeholders. The Cleanverse App ID/API key pasted from the bounty page goes in `.env` only.

Status key:  confirmed in docs this session ·  from memory, confirm in Phase 0 ·  could not be checked (gated or unreachable)

1. Toolchain (local machine, Windows 11)

Tool	Version	Why	Get it
Node.js	≥ 20 LTS	SDKs, middleware, apps	nodejs.org
pnpm	≥ 9	monorepo	<code>npm i -g pnpm</code>
Foundry (forge, cast, anvil)	latest	contracts, tests, deploy	<code>foundryup</code> (run in WSL/Git Bash; or use the Windows binary from <code>getfoundry.sh</code>)
Circom 2 + snarkjs	latest	RLN-v2 circuit params/witness	<code>npm i -g snarkjs</code> ; circom binary from the Circom releases (or reuse prebuilt RLN artifacts, preferred)
Git + GitHub account	—	public repo	repo must be readable by <code>metropolis@hackathon.monad.xyz</code>
A browser with a platform passkey	—	WebAuthn demo	Chrome/Edge with Windows Hello, or Safari/iCloud Keychain, or a phone

Hardware note: Windows Hello works for WebAuthn assertions we verify on-chain. (It lacks the PRF extension. We deliberately do **not** depend on PRF.)

2. Monad

[illegible]

Item	Value	Status
ERC-8004 ReputationRegistry	<code>0x8004BAa17C55a88189AE136b182e5fdA19dE9b63</code>	✅ (mainnet)
ERC-8004 ValidationRegistry address	find in <code>github.com/erc-8004/erc-8004-contracts</code>	?
BN254 precompiles (Groth16 verify)	ecAdd/ecMul/ecPairing present	✅
BTX (encrypted mempool)	public testnet access, interface, whether it hides target + calldata	? Phase 0 gate: ask Monad devrel/mentors on the hackathon Discord
Test MON	faucet (testnet)	⚠️ from the Monad developer portal
Deploy key	a fresh throwaway EOA, funded with test MON only	generate with <code>cast wallet new</code>

Monad has **no global public mempool** (txs go to the next 3 leaders), so the realistic front-runner is a leader or RPC operator. Say this in docs, not "anyone in the mempool."

3. Sponsor accounts and keys

3.1 Privy (agent wallets) — Core

What	How to get it
Account + app	dashboard.privy.io → create app. Gives <code>PRIVY_APP_ID</code> , <code>PRIVY_APP_SECRET</code> .
Authorization key (our runtime signer)	Dashboard → Wallet infrastructure → Authorization keys → New key. Save the private key immediately; Privy never sees it. → <code>PRIVY_AUTH_PRIVATE_KEY</code> , <code>PRIVY_AUTH_KEY_QUORUM_ID</code> .
Policies	created via API/dashboard: allowlist QuotaRegistry; value cap.
Monad	Privy supports Monad ✅. Subsidized testnet usage: email <code>monad@privy.io</code> ✅.
Packages	<code>@privy-io/react-auth</code> (console), <code>@privy-io/node</code> (server)
Phase 0 checks	(a) is <code>personal_sign</code> deterministic for the same message? (b) can policies cover <code>personal_sign</code> /typed data, or only transactions? (c) sign-only (<code>eth_signTransaction</code>) available on Monad.

3.2 Dynamic (service wallets) — Core, with cut rule

What	How to get it
Account + environment	app.dynamic.xyz → create project, sandbox env. Gives <code>DYNAMIC_ENVIRONMENT_ID</code> .
API token (server wallets / delegated access)	Dashboard → Developer → API tokens → <code>DYNAMIC_API_TOKEN</code> .

What	How to get it
Delegated access	Enable in dashboard; configure webhook endpoint + RSA key per docs  . Delegation credentials arrive encrypted at a webhook, so we need a publicly reachable HTTPS endpoint (use a tunnel in dev, deploy for demo).
Custom EVM chain (Monad)	add Monad as a custom EVM network in the dashboard  confirm; Monad lists Dynamic as a supported embedded-wallet provider  .
Packages	<code>@dynamic-labs/sdk-react-core</code> , <code>@dynamic-labs-wallet/node-evm</code> (names  confirm in docs)
Phase 0 check	one successful signed Monad testnet tx from a Dynamic wallet, sign-only. Fail → drop Dynamic.

3.3 Qwen 3.8 Max (Alibaba Cloud) — Core (demo agent)

What	How to get it
Account + API key	qwencloud.com (try-ai) → create key → <code>QWEN_API_KEY</code> . The \$5k credits are awarded to winners; develop on the free/trial quota.
Model ID and endpoint	 confirm exact ID for "Qwen 3.8 Max" and the OpenAI-compatible base URL in the console → <code>QWEN_MODEL</code> , <code>QWEN_BASE_URL</code> . Keep the loop provider-agnostic.
Required deliverable	a published article on how Qwen was used and what value it brought (Medium/dev.to/Hashnode). Draft during Phase 7.

3.4 Nansen (Screener) — Conditional

What	How to get it
API key	docs.nansen.ai → API access. → <code>NANSEN_API_KEY</code> . Check free credits or hackathon credits (ask in the Metropolis sponsor channel). The docs host failed a TLS check from this environment  , so verify in a browser.
Coverage	Monad supported for Smart Money, Token God Mode, Profiler  ; Monad onboarded 14 May 2025, so history starts then  .
Endpoints to use	Profiler: labels, related wallets, counterparties, funding. Exact paths  read the Profiler reference.
Optional	Nansen MCP / CLI. Not needed.

3.5 Cleanverse (CVI/CVA) — Conditional, Phase 0 gate

What	How to get it
Docs	docs.cleanverse.com is invitation-code gated  . Use the access invitation code on the bounty page.
Credentials	App ID + API key from the bounty page → <code>.env</code> as <code>CLEANVERSE_APP_ID</code> , <code>CLEANVERSE_API_KEY</code> . Do not paste into docs, commits, issues, or chat logs.
Integration guides	three Google Drive PDFs linked on the bounty page (CCP CVI Compliance, Wrapped CVA, CVA). Read them in Phase 0.

What	How to get it
Gate (all must pass)	(a) a CVA token exists on Monad testnet; (b) CVI verification is callable on-chain by our contract; (c) sandbox credentials work end to end.

3.6 Monad Foundation — Community Team

What	How
Eligibility	Hackathon portal profile → community field → pick your onboarded campus group. Cannot be added afterward without the profile. Tell me which community, or we skip.

4. Cryptographic artifacts

Artifact	Source	Notes
RLN-v2 circuit, wasm, zkey, verification key	github.com/privacy-ethereum/rln / rlnjs / rln-docs	pin versions and SHA-256 checksums in the repo; use their ceremony output, don't run our own setup
Poseidon (on-chain)	poseidon-solidity or @zk-kit LeanIMT/IMT Solidity libs	measure insert gas on Monad
Merkle tree depth	20 ($\approx 1M$ leaves) to start	smaller depth = faster proofs; decide in Phase 2
WebAuthn parsing (on-chain)	write our own minimal lib, informed by Daimo/Coinbase WebAuthn libs	scored heavily, so full test vectors

5. `.env.example` (committed) — placeholders only

```
# --- Chain ---
MONAD_RPC_URL=
MONAD_CHAIN_ID=
DEPLOYER_PRIVATE_KEY=          # throwaway, testnet-funded only
QUOTA_REGISTRY_ADDRESS=        # filled after deploy

# --- Privy (agent side) ---
PRIVY_APP_ID=
PRIVY_APP_SECRET=
PRIVY_AUTH_PRIVATE_KEY=
PRIVY_AUTH_KEY_QUORUM_ID=

# --- Dynamic (service side) ---
DYNAMIC_ENVIRONMENT_ID=
DYNAMIC_API_TOKEN=
DYNAMIC_WEBHOOK_PUBLIC_URL=

# --- Qwen ---
QWEN_API_KEY=
QWEN_BASE_URL=
QWEN_MODEL=

# --- Nansen (Phase 6, optional) ---
NANSEN_API_KEY=

# --- Cleanverse (Phase 0 gate, optional) ---
CLEANVERSE_APP_ID=
CLEANVERSE_API_KEY=

# --- App ---
WEBAUTHN_RP_ID=localhost
WEBAUTHN_ALLOWED_ORIGINS=http://localhost:3000
```

6. Repo layout (monorepo)

```
/contracts      Foundry: QuotaRegistry, PasskeyAuth, SubmitPath, tests
/packages/client @quota/client
/packages/server @quota/server (middleware + MCP wrapper)
/packages/slasher @quota/slasher
/packages/wallets Privy / Dynamic / Local adapters
/packages/screener Nansen screener (Phase 6)
/apps/console    operator + service consoles (Next.js)
/apps/scout      Qwen agent + violation script
/apps/demo-mcp   reference MCP server behind QUOTA
/docs            PRD, requirements, phases, threat model
```

7. Things only you can do (cannot be automated here)

1. Create the Privy, Dynamic, Qwen, and Nansen accounts and copy the keys into `.env`.

2. Provide the Cleanverse invitation code access and read the Drive guides, or confirm it's fine to drop that bounty.
3. Tell me your **community group** (or skip the community bounty).
4. Email `monad@privy.io` for subsidized testnet use, and ask Monad mentors about **BTX testnet access**.
5. Line up **one named external integrator**. This is worth more than any extra feature (Founder/Market 25% + Traction 20%).
6. Create the public GitHub repo and grant `metropolis@hackathon.monad.xyz` access.